

Sumário

PenguinLab — Documentação Técnica	2
1. Visão geral	2
2. Decisões de arquitetura	2
3. Estrutura de arquivos	3
4. O <code>provision-penguinlab.sh</code> em detalhe	3
5. Ansible	5
6. Scripts auxiliares	5
7. Modelo de segurança	6
8. Política de senhas (decisões registradas)	7
9. Rede: cenários e implicações técnicas	7
10. Problemas encontrados e corrigidos	7
11. Limitações conhecidas	9
12. Verificação e diagnóstico	9
13. Referências de estudo	9

PenguinLab — Documentação Técnica

Referência completa do projeto PenguinLab: decisões de arquitetura, funcionamento interno de cada script, modelo de segurança, problemas conhecidos e materiais de estudo. Para o passo a passo de implantação, consulte o **IMPLANTACAO.md**; para a visão geral, o **README.md**.

1. Visão geral

O PenguinLab é um sistema de provisionamento para laboratórios de informática escolares. Ele aplica três camadas de restrição em cada máquina, sobre uma instalação padrão e não modificada do Linux Mint (ou Ubuntu):

Camada	O que faz
DNS filtrado	Cloudflare for Families (1.1.1.3 / 1.0.0.3) via <code>systemd-resolved</code> , com DNS-over-TLS (<code>opportunistic</code>)
Firefox restrito	Políticas empresariais: extensões bloqueadas, navegação privada desabilitada, DNS-over-HTTPS travado, ferramentas de desenvolvedor bloqueadas, telemetria desativada
Usuário sem privilégios	Usuário <code>aluno</code> com shell <code>bash</code> normal, mas sem <code>sudo</code> , sem ações administrativas via <code>polkit</code> , com login SSH remoto bloqueado e auto-login no LightDM

2. Decisões de arquitetura

Por que não gerar uma ISO?

A abordagem anterior (`live-build` + `squashfs` + `GRUB/isolinux`) era frágil demais: problemas com `kernel panic` no boot, pacotes essenciais removidos pelo `autoremove` e complexidade desproporcional ao escopo. A substituição por um script de provisionamento foi motivada por:

- **Simplicidade:** roda sobre uma instalação padrão, sem build customizado.
- **Idempotência:** pode ser executado várias vezes sem duplicar configurações.
- **Atualização fácil:** para mudar a configuração, basta rodar o script de novo.
- **Escala:** o Ansible aplica o mesmo script em 14 máquinas em paralelo.

Por que `DNSOverTLS=opportunistic` e não `yes`?

`opportunistic` tenta usar DNS-over-TLS e, se a rede bloquear a porta 853 (alguns firewalls/proxies de escola), degrada para DNS comum em vez de quebrar a resolução. A filtragem de conteúdo continua valendo nos dois casos (ela é feita na camada de DNS, independente do TLS).

Por que `polkit` em formato JS?

O Ubuntu 24.04+ não instala `polkitd-pkla` por padrão — arquivos `.pkla` são **silenciosamente ignorados**. O formato moderno de regras (`/etc/polkit-1/rules.d/*.rules`, JavaScript) é o suportado atualmente e é o que o script usa.

3. Estrutura de arquivos

```
PenguinLab/
|-- provision-penguinlab.sh      # Script principal (idempotente, roda como root)
|-- ansible/
|   |-- playbook.yml            # Aplica o script via SSH nas máquinas
|   |-- inventory.ini           # Inventário (14 máquinas de exemplo)
|   |-- bootstrap-ssh.sh        # Descobre IPs e prepara o SSH em lote
|   |-- check-status.sh         # Verifica o estado de provisão remotamente
|   '-- ansible.cfg             # forks=14, timeout, host_key_checking
|-- README.md                   # Visão geral
|-- IMPLANTACAO.md              # Guia prático (com e sem rede)
'-- DOCUMENTACAO.md             # Este arquivo
```

4. O provision-penguinlab.sh em detalhe

4.1 Execução e pré-requisitos

- Deve ser executado como **root** (`sudo bash provision-penguinlab.sh`); a função `check_root` aborta caso contrário.
- Usa `set -euo pipefail`: qualquer erro interrompe a execução com mensagem clara.
- Grava o log em `/var/log/penguinlab-provision.log` (append a cada execução).
- É **idempotente** — reprovisionar não duplica configurações.

4.2 Funções

Função	O que faz
<code>check_root()</code>	Verifica se o script roda como root; aborta com instrução caso contrário
<code>init_log()</code>	Cria o diretório e abre o log com um cabeçalho datado
<code>setup_dns()</code>	Ativa o <code>systemd-resolved</code> (se necessário), escreve <code>/etc/systemd/resolved.conf</code> com os DNS filtrados e <code>DNSOverTLS=opportunistic</code> (fazendo backup do original em <code>resolved.conf.penguinlab.bak</code>), garante que <code>/etc/resolv.conf</code> aponte para o stub do <code>systemd-resolved</code> , e impede o <code>NetworkManager</code> de sobrescrever o DNS via <code>/etc/NetworkManager/conf.d/99-penguinlab-dns.conf</code>
<code>setup_firefox()</code>	Verifica se o Firefox existe (e se é snap), e escreve <code>/etc/firefox/policies/policies.json</code> com todas as restrições

Função	O que faz
<code>setup_user()</code>	Cria o usuário aluno (idempotente), aplica a senha em toda execução , garante o shell bash, remove dos grupos <code>sudo/admin/wheel</code> , bloqueia sudo via <code>/etc/sudoers.d/penguinlab-aluno</code> (validado com <code>visudo</code>) e aplica regras polkit JS
<code>setup_ssh_restriction()</code>	Escreve <code>/etc/ssh/sshd_config.d/90-penguinlab-no-ssh-aluno.conf</code> com <code>DenyUsers aluno</code> , valida com <code>sshd -t</code> e recarrega o serviço
<code>setup_autologin()</code>	Configura o auto-login do aluno em <code>/etc/lightdm/lightdm.conf</code> (seção <code>[Seat:*)</code>) e adiciona o usuário ao grupo <code>autologin</code> (quando o grupo existe)
<code>summary()</code>	Imprime o resumo final do que foi aplicado
<code>main()</code>	Orquestra: <code>check_root -> init_log -> setup_dns -> setup_firefox -> setup_user -> setup_ssh_restriction -> setup_autologin -> summary</code>

4.3 Variáveis de configuração

Variável	Padrão	Descrição
<code>LOGFILE</code>	<code>/var/log/penguinlab-provisioning/</code>	Caminho do log
<code>USUARIO</code>	<code>aluno</code>	Nome do usuário limitado
<code>SENHA_ALUNO</code>	<code>PENGUINLAB_PASSWORD</code> (ou <code>aluno</code>)	Senha do usuário limitado
<code>DNS_PRIMARIO</code>	<code>1.1.1.3</code>	Cloudflare for Families (primário)
<code>DNS_SECUNDARIO</code>	<code>1.0.0.3</code>	Cloudflare for Families (secundário)
<code>HOMEPAGE</code>	<code>https://www.google.com.br</code>	Homepage travada no Firefox

Aviso de senha padrão: se `PENGUINLAB_PASSWORD` não for definida, o script usa `aluno` como senha e imprime um aviso bem visível — comportamento esperado neste projeto, que adota senha uniforme `aluno` para facilitar o uso pelas crianças (a conta não tem privilégios; ver seção 8).

4.4 Arquivos criados/alterados em cada máquina

Arquivo	Camada
<code>/etc/systemd/resolved.conf (+ .penguinlab.bak)</code>	DNS
<code>/etc/NetworkManager/conf.d/99-penguinlab-dns.conf</code>	DNS
<code>/etc/firefox/policies/policies.json</code>	Firefox
<code>/etc/sudoers.d/penguinlab-aluno</code>	Usuário
<code>/etc/polkit-1/rules.d/90-penguinlab-restrict.rules</code>	Usuário
<code>/etc/ssh/sshd_config.d/90-penguinlab-no-ssh-aluno.conf</code>	SSH
<code>/etc/lightdm/lightdm.conf</code>	Auto-login

Arquivo	Camada
---------	--------

5. Ansible

5.1 playbook.yml

Quatro tarefas, todas com `become: true` (executam como root):

1. **Copiar o script** -> `provision-penguinlab.sh` para `/tmp/` (modo 0755).
2. **Executar** -> `/tmp/provision-penguinlab.sh` com `PENGUINLAB_PASSWORD` no ambiente da tarefa (padrão: `aluno`).
3. **Exibir resultado** -> imprime a saída do script no terminal do notebook (via `debug`).
4. **Remover o temporário** -> apaga `/tmp/provision-penguinlab.sh`.

A última tarefa (reiniciar as máquinas) está comentada por padrão — o reinício fica sob controle do administrador.

5.2 inventory.ini

- Grupo `[penguinlab]` com 14 hosts de exemplo (`penguinlab-01...penguinlab-14`, IPs `192.168.0.21-192.168.0.34`).
- `[all:vars]` -> `ansible_user=professor` (a conta administrativa — **nunca** `aluno`).
- Os IPs de exemplo devem ser substituídos pelos reais do laboratório.

5.3 ansible.cfg

```
[defaults]
forks = 14           # provisão das 14 máquinas em paralelo
host_key_checking = False
timeout = 15
```

6. Scripts auxiliares

6.1 bootstrap-ssh.sh

Prepara as máquinas após a instalação manual do sistema. Executado no notebook (máquina de controle).

- **Descoberta:** opção `--descobrir <faixa>` varre uma sub-rede `/24` testando a porta 22 via TCP (`echo >/dev/tcp/<ip>/22`), com até 50 conexões concorrentes. Alternativamente, os IPs podem ser passados diretamente como argumentos.
- **Checagem de SSH:** antes de qualquer comando remoto, testa a porta 22 de cada host. Se fechada (comum em instalações novas, onde o `sshd` vem desativado), imprime a orientação completa (`sudo apt install -y openssh-server && sudo systemctl enable --now ssh`), **pula a máquina e continua o lote**.
- **Preparação:** garante `openssh-server` e `python3` (esforço máximo, sem falha fatal), copia a chave pública do notebook para a conta `professor` (via `ssh-copy-id`) e valida o login sem senha.
- **Chave:** gera automaticamente `~/.ssh/id_ed25519` se ainda não existir.

- **Configuração:** USUARIO_ADMIN (padrão professor, via PENGUINLAB_ADMIN), log em bootstrap-ssh.log.

6.2 check-status.sh

Verifica remotamente se cada máquina está provisionada. Usa o inventory.ini (extraindo ansible_host) ou IPs passados como argumentos.

Checagens por máquina: 1. Conexão SSH e existência do usuário aluno. 2. aluno fora dos grupos sudo/admin/wheel. 3. Sudo bloqueado para aluno (procura “not allowed”/!ALL). 4. DNS filtrado presente em /etc/systemd/resolved.conf. 5. policies.json do Firefox presente. 6. Regras polkit presentes. 7. Auto-login do aluno configurado no LightDM. 8. Bloqueio de SSH remoto do aluno presente.

Saída com [OK]/[FALHA] por checagem e resumo final (X/Y maquinas totalmente OK).

7. Modelo de segurança

O que o aluno pode fazer

- Usar o desktop normalmente (navegador restrito, aplicativos comuns, terminal **livre** — o shell não é restrito).
- Salvar arquivos na própria home (/home/aluno).

O que o aluno não pode fazer

Ação	Mecanismo de bloqueio
sudo / virar root	/etc/sudoers.d/penguinlab-aluno (aluno ALL=(ALL) !ALL)
Instalar pacotes	Regra polkit (prefixo org.freedesktop.packagekit.)
Alterar rede/Wi-Fi	Regra polkit (prefixo org.freedesktop.NetworkManager.)
Montar discos/pendrives	Regra polkit (prefixo org.freedesktop.udisks2.)
Gerenciar serviços systemd	Regra polkit (prefixo org.freedesktop.systemd1.)
Login SSH remoto	DenyUsers aluno no sshd

A conta professor

Não é alterada pelo provisionamento (apenas perde o auto-login, se existisse). Mantém sudo e acesso total. É a conta usada pelo Ansible — por isso a senha dela deve ser forte e **nunca** igual à do aluno.

Auto-login

O aluno entra automaticamente ao ligar a máquina. Como consequência, a senha do aluno só é usada no desbloqueio de tela (se o bloqueio estiver ativado) e em logouts manuais.

8. Política de senhas (decisões registradas)

Conta	Senha	Justificativa
aluno	aluno, uniforme em todas as máquinas	Facilidade para crianças digitarem no desbloqueio de tela. Risco aceitável: a conta é totalmente sem privilégios, sem SSH remoto, e o DNS/navegador continuam filtrados
professor	Forte, igual em todas as máquinas	É a conta com sudo; uma única senha forte é mais simples de gerenciar. Deve ser guardada com cuidado

- A senha do **aluno** é aplicada em **toda** execução do script (não só na criação do usuário), portanto reprovisionar com outro valor a atualiza.
- O `PENGUINLAB_PASSWORD` precisa ser passado ao script via ambiente; no Ansible, o play-book o propaga via `environment::`; no pendrive, deve ser passado junto do `sudo` (`sudo PENGUINLAB_PASSWORD='...' bash ...`), pois o `sudo` limpa variáveis de ambiente por padrão.

9. Rede: cenários e implicações técnicas

- **Mesmo roteador (wifi e cabo na mesma sub-rede):** o notebook no wifi alcança as máquinas cabeadas normalmente; o `--descobrir` e o Ansible funcionam sem ajustes.
- **Sub-redes separadas (ou VLANs com firewall):** o notebook não alcança as máquinas — nem o `--descobrir` (ele varre uma única faixa /24), nem o SSH. Soluções: plugar o notebook no cabo, usar um travel router em modo bridge, ou adotar o provisionamento por pendrive.
- **Porta 853 (DNS-over-TLS):** se a rede da escola bloquear a porta 853, o DNS muda (via `opportunistic`) para DNS comum — a filtragem de conteúdo continua, pois é feita na camada de resolução.
- **AP isolation / guest networks:** em redes com isolamento de cliente, mesmo na mesma faixa o tráfego entre dispositivos pode ser bloqueado; nesse caso, o notebook deve entrar na rede via cabo/travel router.

10. Problemas encontrados e corrigidos

Problema	Severidade	O que acontecia	Correção
Self-lockout no Ansible	Crítico	<code>ansible_user=aluno</code> + <code>!ALL</code> no sudo travava o Ansible após a primeira execução	Documentar <code>ansible_user=professor</code> como obrigatório

Problema	Severidade	O que acontecia	Correção
<code>polkit .pkla</code> ignorado	Crítico	Ubuntu 24.04 não tem <code>polkitd-pkla</code> ; regras <code>.pkla</code> eram ignoradas silenciosamente	Migrar para regras JS em <code>/etc/polkit-1/rules.d/</code>
Chaves JSON duplicadas	Moderado	<code>DisableSystemAddonUpdate</code> , <code>DisableTelemetry</code> etc. duplicados no <code>policies.json</code>	Reorganizar e remover duplicatas
Log sobrescrito	Menor	<code>init_log()</code> usava <code>></code> (sobrescrevia)	Trocar para <code>>></code> (append)
Chave SSH sobrescrita no bootstrap	Crítico	<code>ssh-keygen</code> respondia “y” ao prompt de sobrescrita, invalidando acessos existentes	Gerar chave apenas se ainda não existir
Checagem de grupos admin nunca rodava	Alto	Condição externa morta no <code>check-status.sh</code>	Remover o <code>if</code> externo
Doc de autologin desatualizada	Moderado	Doc citava <code>slick-greeter.conf</code> ; script usa <code>lightdm.conf</code>	Alinhar documentação ao código
SSH remoto do aluno liberado	Alto (segurança)	aluno podia autenticar via SSH remotamente	Nova função <code>setup_ssh_restriction()</code> com <code>DenyUsers</code>
Senha padrão previsível	Moderado (segurança)	Todas usavam <code>aluno/aluno</code> se <code>PENGUINLAB_PASSWORD</code> não definida	Aviso visível + documentação exigindo senha adequada em produção
<code>resolved.conf</code> sobrescrito sem backup	Baixo	Configuração DNS original perdida permanentemente	Backup automático antes de sobrescrever
Detecção de polkit falha (<code>dpkg -l</code>)	Baixo	<code>dpkg -l</code> retornava 0 mesmo com pacote removido	Trocar para <code>dpkg -s</code> com checagem de status
Senha do aluno não atualizada em re-execução	Moderado	<code>chpasswd</code> rodava apenas na criação do usuário	Aplicar <code>chpasswd</code> em toda execução do <code>setup_user</code>
Bootstrap cego quando <code>sshd</code> inativo	Alto (fluxo)	Instalação nova não ativa <code>sshd</code> ; bootstrap falhava de forma genérica ou nem achava a máquina	Checar a porta 22 com orientação explícita (<code>apt/systemctl</code>) e pular a máquina

11. Limitações conhecidas

- **Bypass de DNS via outros navegadores/AppImages:** o filtro de DNS e as políticas do Firefox **não** impedem que o aluno baixe um AppImage de outro navegador com DNS-over-HTTPS embutido, contornando a filtragem na camada de rede. A mitigação completa (firewall bloqueando endpoints/portas de DoH conhecidos) está **fora do escopo**.
 - **Firefox via snap (Ubuntu 24.04+):** o sandboxing do snap pode fazer com que o `policies.json` em `/etc/firefox/policies/` seja ignorado. O script detecta e avisa. Solução: trocar para o `.deb` (`sudo snap remove firefox && sudo apt install firefox`) — o Linux Mint já vem com o Firefox `.deb`, por isso é a distribuição recomendada.
 - **Sem reset de sessão:** não há limpeza automática de `/home/aluno` ao religar — os arquivos do aluno persistem.
 - **polkit ausente:** se `polkitd` não estiver instalado, o script avisa e segue (as restrições `polkit` não são aplicadas nessa máquina, sem erro fatal).
 - **Shell do aluno é livre:** o provisionamento não restringe o terminal. Para crianças muito pequenas, uma limitação adicional seria uma evolução futura.
-

12. Verificação e diagnóstico

- **Em lote (com rede):** `ansible/check-status.sh` — 8 checagens por máquina com resumo final.
 - **Manual:** tabela da seção 7 do **IMPLANTACAO.md** (DNS, Firefox, sudo, SSH, polkit, auto-login).
 - **Logs:** `/var/log/penguinlab-provision.log` em cada máquina; `bootstrap-ssh.log` no notebook.
 - **Problema comum:** máquina não aparece no `--descobrir` -> quase sempre é o `sshd` desativado (execute o passo 3.2 do **IMPLANTACAO** e rode o `bootstrap` de novo).
-

13. Referências de estudo

Linux / Systemd

- `systemd-resolved` — <https://www.freedesktop.org/software/systemd/man/systemd-resolved.html>
- `NetworkManager + systemd-resolved` — <https://developer.gnome.org/NetworkManager/stable/nm-dns.html>

Shell Scripting (Bash)

- `Advanced Bash-Scripting Guide` — <https://tldp.org/LDP/abs/html/>
- `Bash Reference Manual` — <https://www.gnu.org/software/bash/manual/bash.html>

Políticas Firefox Enterprise

- `Enterprise Policy documentation` — <https://mozilla-policy.readthedocs.io/en/latest/>
- `policies.json reference` — https://mozilla-policy.readthedocs.io/en/latest/policies_json/

polkit (PolicyKit)

- `polkit documentation` — <https://www.freedesktop.org/wiki/Software/polkit/>

- polkit rules (formato JS) — <https://www.freedesktop.org/software/polkit/docs/latest/polkit.8.html>

Ansible

- Documentação oficial — <https://docs.ansible.com/ansible/latest/index.html>
- Playbooks — https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_intro.html
- Inventário — https://docs.ansible.com/ansible/latest/inventory_guide/index.html
- Escalação de privilégios (become) — https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_privileges.html

Linux Mint / LightDM / Slick Greeter

- LightDM — <https://wiki.lightdm.org/>
- Slick Greeter — <https://github.com/linuxmint/slick-greeter>

Redes e DNS

- Cloudflare for Families — <https://developers.cloudflare.com/1.1.1.1/>
- DNS-over-TLS — <https://developers.cloudflare.com/1.1.1.1/dns-over-tls/>
- RFC 8484 — DNS Queries over HTTPS — <https://datatracker.ietf.org/doc/html/rfc8484>

Documentação técnica do PenguinLab. Toda decisão de design, problema encontrado e correção aplicada está registrada neste arquivo.